

S.N. Polymers

Integrated Digital Business Platform

PHASE 1

Full Codebase Documentation

Document Type: Technical Reference & Developer Onboarding Guide

Scope: Authentication, RBAC, Session Management, Admin Console

Last Updated: 4th June 2026

Intern Development Batch 2026

Table of Contents

1. [Repository Layout](#)
2. [Technology Stack & Dependencies](#)
3. [Database Schema \(Supabase/PostgreSQL\)](#)
4. [Backend — backend/](#)
 - 4.1 [Entry Point — app.js](#)
 - 4.2 [Database Client — db/supabase.js](#)
 - 4.3 [Routes](#)
 - 4.4 [Controllers](#)
 - 4.5 [Services](#)
 - 4.6 [Middleware](#)
 - 4.7 [Environment Configuration — .env.example](#)
5. [Frontend — frontend/](#)
 - 5.1 [Build Configuration — vite.config.js](#)
 - 5.2 [Design System — tailwind.config.js](#)
 - 5.3 [Application Root — main.jsx & App.jsx](#)
 - 5.4 [API Client — api/authApi.js](#)
 - 5.5 [Components](#)
 - 5.6 [Pages](#)
6. [Request Lifecycle Walkthroughs](#)
7. [Security Model](#)
8. [Deployment Architecture](#)

1. Repository Layout

The monorepo is organized into two top-level packages — a Node.js/Express backend and a React/Vite frontend — alongside documentation files at the root.

1.1 Directory Structure

SNPolymers/	← Monorepo root
— backend/	← Node.js + Express REST API
— src/	
— app.js	← Express server entry point
— db/	
— supabase.js	← Supabase client singleton
— routes/	
— auth.routes.js	← Public + authenticated auth
endpoints	
— admin.routes.js	← Admin-only CRUD endpoints
— controllers/	
— auth.controller.js	← OTP, verify, logout, /me
handlers	
— admin.controller.js	← User whitelist + sessions CRUD
— services/	
— otp.service.js	← OTP generation, hashing,
verification	
— whatsapp.service.js	← Twilio WABA message delivery
— session.service.js	← JWT issuance + session DB CRUD
— email.service.js	← Admin login/logout email alerts
— middleware/	
— verifyJwt.js	← JWT cookie guard (all protected
routes)	
— requireAdmin.js	← Role guard (admin-only routes)
— rateLimiter.js	← express-rate-limit configs
— .env	← Local secrets (gitignored)
— .env.example	← Template with all required keys
— package.json	
— frontend/	← React 19 + Vite SPA
— public/assets/logo.png	← Company logo asset
— index.html	← Vite HTML shell

```
| | | vite.config.js           ← Vite build configuration
| | | tailwind.config.js      ← Custom design token palette
| | | postcss.config.js       ← PostCSS pipeline
| | | └─ src/
| | |   |─ main.jsx           ← React root mount
| | |   |─ App.jsx            ← Route tree definition
| | |   |─ index.css          ← Tailwind directives
| | |   |─ api/
| | |   |   └─ authApi.js      ← Axios instance (credentials +
base URL)
| | |   |─ components/
| | |   |   |─ AuthContext.jsx  ← Global auth state provider
| | |   |   |─ ProtectedRoute.jsx ← Route guards (role-aware)
| | |   |   └─ pages/
| | |   |       |─ Home.jsx     ← Public landing / corporate
gateway
| | |   |       |─ Login.jsx    ← Mobile number entry + OTP
request
| | |   |       |─ OtpVerify.jsx ← 6-digit OTP input + timer
| | |   |       |─ Dashboard.jsx ← Post-login console (staff +
admin)
| | |   |       └─ admin/
| | |   |           |─ AdminPanel.jsx ← Whitelist management CRUD UI
| | |   |           |─ AuditLog.jsx  ← Session history with filters
| | |
| | | └─ CODEBASE_DOCUMENTATION.md ← This file
| | | └─ PHASE1_DOCUMENTATION.md   ← High-level phase summary
| | | └─ implementation_plan.md     ← Original design blueprint
| | | └─ README.md                  ← Setup & deployment guide
| | | └─ .gitignore
```

2. Technology Stack & Dependencies

2.1 Backend Dependencies

Package	Version	Purpose
express	^4.19.2	HTTP server and routing framework

Package	Version	Purpose
@supabase/supabase-js	^2.43.4	Supabase DB client (PostgreSQL via REST)
jsonwebtoken	^9.0.2	JWT signing and verification (HS256)
bcrypt	^6.0.0	OTP hash generation and comparison
uuid	^14.0.0	Unique session JTI generation (v4)
cookie-parser	^1.4.6	Parses incoming httpOnly token cookies
cors	^2.8.5	Cross-Origin Resource Sharing headers
helmet	^8.2.0	Security-hardening HTTP headers
dotenv	^16.4.5	Environment variable loading from .env
express-rate-limit	^7.3.1	IP/mobile-level request throttling
twilio	^5.1.0	WhatsApp Business API (WABA) OTP delivery
nodemailer	^8.0.10	Gmail SMTP email alert transport
nodemon (dev)	^3.1.2	Hot-reload development server

NPM Scripts

```
npm run dev    # nodemon src/app.js (development with hot-reload)
npm start     # node src/app.js      (production)
```

2.2 Frontend Dependencies

Package	Version	Purpose
react	^19.2.6	UI component library
react-dom	^19.2.6	React DOM renderer
react-router-dom	^6.23.1	Client-side routing (v6 data router API)
axios	^1.7.2	HTTP client with withCredentials support
tailwindcss	^3.4.4	Utility-first CSS framework
vite	^8.0.12	Development server and production bundler
@vitejs/plugin-react	^6.0.1	JSX transform + Fast Refresh

Package	Version	Purpose
postcss + autoprefixer	^8.4 / ^10.4	CSS build pipeline

NPM Scripts

```
npm run dev      # Vite dev server on :5173
npm run build    # Production bundle output to dist/
npm run preview  # Serve production build locally
npm run lint     # ESLint check
```

3. Database Schema (Supabase / PostgreSQL)

Three tables are defined in Supabase. All are protected by Row Level Security (RLS). The backend accesses them exclusively via the Service Role Key, bypassing RLS for all server-side operations.

3.1 Table: authorised_users

Stores the mobile-number whitelist of personnel permitted to log in.

```
CREATE TABLE authorised_users (  
  id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),  
  mobile_number varchar(15) UNIQUE NOT NULL, -- E.164 format: +919876543210  
  display_name varchar(100),  
  role         varchar(50)  DEFAULT 'staff', -- 'staff' | 'admin'  
  permissions  jsonb        DEFAULT '{}',    -- reserved for Phase 2  
  created_at   timestampz   DEFAULT now(),  
  is_active    boolean      DEFAULT true  
);
```

Key Behaviors

- `is_active = false` blocks login even if the mobile number matches.
- `role = 'admin'` grants access to `/admin/*` routes.
- `permissions` (JSONB) is reserved for future module-level access flags (e.g., `{"manufacturing": true}`).

3.2 Table: otp_requests

Stores one bcrypt-hashed OTP record per send event. Never stores plaintext.

```
CREATE TABLE otp_requests (  
  id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),  
  mobile_number varchar(15) NOT NULL,  
  otp_hash     text          NOT NULL,      -- bcrypt hash, NOT the raw code  
  expires_at   timestampz   NOT NULL,      -- now() + 5 minutes  
  is_used      boolean      DEFAULT false,  
  attempts     int          DEFAULT 0,      -- incremented on wrong guess  
  created_at   timestampz   DEFAULT now()  
);
```

Key Behaviors

- Only the latest record with `is_used = false` for a given mobile is evaluated.
- If `attempts >= 3`, the code is dead (locked) — user must request a new OTP.
- If `now() > expires_at`, the code is expired — user must request a new OTP.

- On success, `is_used` is set to true before the session is created.

3.3 Table: sessions

Complete audit ledger. One row per login event. Updated on logout.

```
CREATE TABLE sessions (  
  id          uuid          PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     uuid          REFERENCES authorised_users(id),  
  jwt_jti     varchar(100) UNIQUE,          -- UUID v4, embedded in JWT  
  ip_address  varchar(45),          -- supports IPv4 and IPv6  
  user_agent  text,  
  login_at    timestamptz  DEFAULT now(),  
  logout_at   timestamptz,          -- null while active  
  duration_seconds int,          -- computed on logout  
  module      varchar(50)  DEFAULT 'office',  
  is_active   boolean      DEFAULT true  
);
```

Key Behaviors

- `jwt_jti` is the linkage between the JWT token and this DB record.
- On every protected API call, `verifyJwt` checks `is_active = true` on this row.
- Logout sets `is_active = false`, `logout_at`, and `duration_seconds`.
- Admin deactivation / deletion instantly sets `is_active = false` on all matching rows.

4. Backend — backend/

4.1 Entry Point — app.js

Responsibility: Express application factory. Registers all global middleware and mounts route handlers.

Step	Code Section	Detail
1	Production guards	If <code>NODE_ENV=production</code> , crashes on missing/default <code>JWT_SECRET</code> or <code>localhost FRONTEND_URL</code> .
2	<code>helmet()</code>	Adds secure HTTP headers (<code>X-Frame-Options</code> , <code>Content-Security-Policy</code> , etc.).
3	<code>cors(corsOptions)</code>	Whitelists only the configured <code>FRONTEND_URL</code> origin. <code>credentials: true</code> required for cookie transport.
4	<code>express.json()</code>	Parses JSON request bodies.
5	<code>cookieParser()</code>	Makes <code>req.cookies.token</code> available to all middleware.
6	<code>trust proxy</code>	Enables accurate IP detection behind Vercel/Render reverse proxies.
7	Route mounting	<code>/api/v1/auth -> auth.routes.js</code> , <code>/api/v1/auth/admin -> admin.routes.js</code>
8	<code>GET /health</code>	Simple liveness check endpoint for monitoring.
9	404 handler	Returns <code>{ success: false, message: 'Resource not found.' }</code>
10	Error handler	Catches unhandled errors, logs stack trace, returns 500.

4.2 Database Client — db/supabase.js

Creates and exports a single shared Supabase client instance using the Service Role Key, which has full database access and bypasses all RLS policies.

```
const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_ROLE_KEY // NOT the anon key
);
module.exports = { supabase };
```

IMPORTANT: Using the Service Role Key means all database access is authenticated with full admin privileges. This is intentional — the Express API is the security boundary. Never expose this key to the client.

4.3 Routes

auth.routes.js — Mounts at /api/v1/auth

```
// Public — no token required
POST /request-otp  <- otpRequestLimiter -> requestOtp
POST /verify-otp   <- otpVerifyLimiter  -> verifyOtpCode

// Protected — requires valid JWT cookie
POST /logout       <- verifyJwt -> logout
GET  /me           <- verifyJwt -> getMe
```

admin.routes.js — Mounts at /api/v1/auth/admin

All routes pass through verifyJwt then requireAdmin before reaching handlers.

```
router.use(verifyJwt);      // applied globally to entire admin router
router.use(requireAdmin);   // role check after token verification

GET    /users             -> getUsers
POST   /users             -> addUser
PATCH /users/:id         -> updateUser
DELETE /users/:id         -> removeUser
GET    /sessions          -> getSessions
```

4.4 Controllers

auth.controller.js

requestOtp(req, res) — POST /api/v1/auth/request-otp

1. Validates mobileNumber is present and matches E.164 regex.
2. Queries authorised_users where mobile_number = \$1 AND is_active = true.
3. If not found: returns 403 Access denied.
4. Generates 6-digit OTP via generateOtp().
5. Hashes it via hashOtp() using bcrypt.
6. Stores hash + expiry via storeOtp().
7. Delivers OTP via sendOtp() (WhatsApp or console log in dev).
8. Returns 200 { success: true }.

verifyOtpCode(req, res) — POST /api/v1/auth/verify-otp

9. Calls verifyOtp(mobileNumber, otp) — returns { success, reason, attemptsLeft }.
10. If failed: returns reason and remaining attempts.

11. Re-fetches user from `authorised_users`.
12. Calls `generateToken(user)` to get `{ token, jti }`.
13. Calls `createSession({ userId, jti, ipAddress, userAgent })`.
14. Sets `res.cookie('token', token, { httpOnly: true, maxAge: 24h })`.
15. Fires `notifyAdminLogin(...)` asynchronously (non-blocking).
16. Returns `200 { success: true, user: { id, mobile_number, display_name, role, permissions } }`.

Cookie Settings by Environment

Environment	httpOnly	secure	sameSite
Development	true	false	lax
Production	true	true	none

`logout(req, res)` — **POST** `/api/v1/auth/logout`

17. Gets `jti` from `req.jti` (set by `verifyJwt`).
18. Calls `closeSession(jti)` — updates DB, returns closed session row.
19. Calls `formatDuration(session.duration_seconds)` for human-readable duration.
20. Calls `res.clearCookie('token', ...)`.
21. Fires `notifyAdminLogout(...)` asynchronously.
22. Returns `200 { success: true }`.

`getMe(req, res)` — **GET** `/api/v1/auth/me`

Returns the `req.user` object attached by `verifyJwt`.

```
{
  "success": true,
  "user": {
    "id": "uuid",
    "mobile_number": "+91XXXXXXXXXX",
    "role": "staff|admin",
    "permissions": {},
    "displayName": "John Doe"
  }
}
```

`admin.controller.js`

All five functions require Admin JWT (enforced at route level).

Function	Route	Description
getUsers	GET /admin/users	Fetches all users, merges session_count and last_login_at per user.
addUser	POST /admin/users	Validates E.164 number, inserts into authorised_users. Handles 23505 UNIQUE violation.
updateUser	PATCH /admin/users/:id	Partial update. If isActive=false, closes all active sessions immediately.
removeUser	DELETE /admin/users/:id	Closes all active sessions, then hard-deletes user.
getSessions	GET /admin/sessions	Joined query with optional userId, dateFrom, dateTo filters.

4.5 Services

otp.service.js

Export	Signature	Description
generateOtp	() -> string	crypto.randomInt(100000, 999999).toString() – cryptographically secure 6-digit code.
hashOtp	(otp) -> Promise<string>	bcrypt.hash(otp, 10) – 10 salt rounds.
storeOtp	(mobile, hash) -> Promise<row>	Inserts into otp_requests with expires_at = now() + 5 min.
verifyOtp	(mobile, rawOtp) -> Promise<result>	Full verification pipeline (see below).

verifyOtp — Detailed Logic

```

1. SELECT latest is_used=false record for mobile, ORDER BY created_at DESC LIMIT 1
2. If none -> { success: false, reason: 'No active OTP request found.' }
3. If expired -> { success: false, reason: 'OTP has expired.' }
4. If attempts >= 3 -> { success: false, reason: 'Too many failed attempts...' }
5. bcrypt.compare(rawOtp, otpRequest.otp_hash)
   - If false -> UPDATE attempts++ -> { success: false, attemptsLeft: 2 - attempts }
   - If true -> UPDATE is_used=true -> { success: true }

```

whatsapp.service.js

Sends OTP via Twilio WABA or falls back to console log in development.

Initialization Logic

```
// Only instantiates Twilio client if BOTH credentials present and non-placeholder
if (accountSid && authToken &&
    accountSid !== 'your_twilio_account_sid' &&
    authToken !== 'your_twilio_auth_token') {
  client = twilio(accountSid, authToken);
}
```

Production Message Body

```
Your Integrated Digital Business Platform (IDBP) security OTP code is: {otp}.
It is valid for 5 minutes. Do not share this code.
```

session.service.js

Export	Signature	Description
generateToken	(user) -> { token, jti }	Signs JWT with HS256, embeds jti = uuidv4().
createSession	({ userId, jti, ip, ua }) -> row	Inserts session row in sessions table.
closeSession	(jti) -> row	Computes duration_seconds, sets logout_at + is_active=false.
formatDuration	(seconds) -> string	Converts integer seconds to HH:MM:SS format string.

JWT Payload Structure

```
{
  "user_id":      "uuid",
  "mobile_number": "+91XXXXXXXXXX",
  "role":         "staff | admin",
  "permissions":  {},
  "jti":          "uuid-v4",
  "iat":          1234567890,
  "exp":          1234567890
}
```

email.service.js

Sends non-blocking HTML email alerts to ADMIN_EMAIL on login and logout events.

Export	Trigger	Description
notifyAdminLogin	Successful OTP verify	Sends login alert with mobile number, role, IP address, and user-agent.
notifyAdminLogout	POST /logout success	Sends logout alert with mobile number, session duration, and logout time.

Note: Both functions use the callback-based nodemailer API. Errors are logged but never thrown, and the HTTP response is never delayed.

4.6 Middleware

verifyJwt.js

Applied to: POST /logout, GET /me, and all /admin/* routes.

Verification Pipeline

```
1. Extract token from req.cookies.token
   -> If missing: 401 'Authentication required. No token provided.'

2. jwt.verify(token, JWT_SECRET)
   -> If invalid/expired: 401 'Authentication failed. Invalid or expired token.'

3. SELECT is_active FROM sessions WHERE jwt_jti = decoded.jti LIMIT 1
   -> If not found or is_active=false: clear cookie -> 401 'Session is inactive...'

4. SELECT is_active FROM authorised_users WHERE id = decoded.user_id LIMIT 1
   -> If not found or is_active=false: 403 'Access denied. Account is deactivated.'

5. Attach to req:
   req.user = { id, mobile_number, role, permissions, displayName }
   req.jti = decoded.jti

6. next()
```

Design Note: The double-check (JWT signature + live DB session status) ensures that a valid token issued to a deactivated or deleted user is still rejected in real time.

requireAdmin.js

Applied to all routes under admin.routes.js (after verifyJwt). Returns 403 if req.user.role !== 'admin'.

rateLimiter.js

Limiter	Applied To	Window	Max Requests	Key
otpRequestLimiter	POST /request-otp	15 min	3	mobileNumber or IP
otpVerifyLimiter	POST /verify-otp	5 min	5	mobileNumber or IP

Key Generator Strategy: Rate limiting prefers mobile number over IP address. This prevents shared-IP environments (offices, NAT) from blocking innocent users when one device exhausts the limit.

4.7 Environment Configuration — .env.example

```
# Server
PORT=5000
NODE_ENV=development
FRONTEND_URL=http://localhost:5173

# Supabase
SUPABASE_URL=your_supabase_project_url
SUPABASE_SERVICE_ROLE_KEY=your_supabase_service_role_key

# JWT
JWT_SECRET=generate_a_secure_at_least_256_bit_secret_string
JWT_EXPIRY=24h

# Twilio WhatsApp
TWILIO_ACCOUNT_SID=your_twilio_account_sid
TWILIO_AUTH_TOKEN=your_twilio_auth_token
TWILIO_WHATSAPP_FROM=whatsapp:+14155238886

# Email Alerts
GMAIL_USER=your_gmail_sender@gmail.com
GMAIL_APP_PASSWORD=your_gmail_app_password
ADMIN_EMAIL=admin_notifications_receiver@domain.com
```

CAUTION: The actual .env file is gitignored. Never commit real secrets. In production, inject these via Render's environment variable dashboard.

5. Frontend — frontend/

5.1 Build Configuration — vite.config.js

```
export default defineConfig({
  plugins: [react()], // JSX transform + Fast Refresh
  server: {
    port: 5173,
    host: true // Binds to 0.0.0.0 (accessible on LAN)
  }
})
```

SPA Routing on Vercel: The vercel.json in the frontend root rewrites all routes to index.html, enabling React Router to handle client-side navigation.

5.2 Design System — tailwind.config.js

The theme extends Tailwind with a custom Admin/Corporate dark palette and Inter typography.

Token	Value	Usage
admin-bg	#0b111e	Page background – deep navy
admin-card	#151c2c	Card/container surfaces
admin-border	#222f47	Dividers, input borders
admin-primary	#1d4ed8	Royal blue accents
admin-accent	#b45309	Amber/gold – secondary CTA, badges
slate-950	#0b0f19	Deepest input backgrounds
Font	Inter (Google Fonts)	System sans-serif throughout

5.3 Application Root — main.jsx & App.jsx

App.jsx defines the full route tree, wrapped in AuthProvider:

```
<AuthProvider>
  <BrowserRouter>
    <Routes>
      / -> <Home /> (public)
      /login -> <Login /> (public)
      /verify-otp -> <OtpVerify /> (public)
      <ProtectedRoute allowedRoles={['staff', 'admin']>
        /dashboard -> <Dashboard />
      <ProtectedRoute allowedRoles={['admin']>
```



```
    /admin                -> <AdminPanel />
    /admin/sessions       -> <AuditLog />
    *                     -> Navigate to /                (404 fallback)
  </Routes>
</BrowserRouter>
</AuthProvider>
```

5.4 API Client — api/authApi.js

```
const authApi = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api/v1/auth',
  withCredentials: true,    // critical: sends the httpOnly token cookie
  headers: { 'Content-Type': 'application/json' }
});
```

withCredentials: true is mandatory for cross-origin cookie transport (Vite on :5173 -> Express on :5000). This also applies in production (Vercel -> Render).

5.5 Components

AuthContext.jsx — Global Auth State Manager

Context value exposed to all pages and route guards:

```
{
  user: UserObject | null,    // null = unauthenticated
  loading: boolean,           // true during initial /me check
  checkAuth: () => void,       // re-runs the /me check
  login: (userData) => void,    // called after successful OTP verify
  logout: () => void           // calls POST /logout, then sets user=null
}
```

Initialization: On first mount, calls GET /api/v1/auth/me. If the browser cookie is valid, user is set from the response. If not (401), user stays null. This is how the app determines auth state across page refreshes.

ProtectedRoute.jsx — Role-Aware Route Guard

```
Loading state  -> renders animated spinner (full-screen dark bg)
No user        -> <Navigate to="/login" replace />
Wrong role     -> <Navigate to="/dashboard" replace />
Authorized     -> <Outlet /> (renders the nested child route)
```

Props: allowedRoles: string[] — e.g., ['admin'] or ['staff', 'admin']

5.6 Pages

Home.jsx — Route: /

Section	Content
Header	Company logo + 'S.N. Polymers' brand + conditional nav button
Hero	'Integrated Digital Business Platform' title with gradient amber text
Divisions	Two info cards: Manufacturing Division & Government Infrastructure Projects
Footer	Copyright, audit notice

Login.jsx — Route: /login

Mobile number entry form with auto-formatting.

```
// Auto-prefix 10-digit Indian numbers
if (/^\d{10}$/.test(formattedNumber)) {
  formattedNumber = `+91${formattedNumber}`;
}
// E.164 validation
if (!/^\+?[1-9]\d{1,14}$/.test(formattedNumber)) { ... }
```

Submit Flow

- 23. Formats number to E.164.
- 24. POST /request-otp with { mobileNumber }.
- 25. On success: navigate('/verify-otp', { state: { mobileNumber } }).
- 26. On failure (403): displays server error message.

OtpVerify.jsx — Route: /verify-otp

6-digit OTP input with full UX controls.

State Variable	Purpose
otp[6]	Array of 6 digit strings, one per input box
countdown	300s (5 min) expiry timer, decrements via setInterval
resendTimer	30s cooldown before 'Request Re-dispatch' appears

State Variable	Purpose
<code>resendDisabled</code>	Controls button visibility
<code>loading, error, success</code>	Submit state feedback

Input Behaviors

- `onChange`: only accepts digits, auto-focuses next box.
- `onKeyDown`: Backspace on empty box focuses previous box.
- `onPaste`: strips non-digits, distributes 6-digit paste across all inputs.

Dashboard.jsx — Route: `/dashboard`

Post-login operator console accessible by Staff and Admin roles.

Module Card	Title	Status
1	Manufacturing Module	Phase 2+ Rollout — Access Restricted
2	Project Management	Phase 2+ Rollout — Access Restricted
3	Office Administration	Active System — links to <code>/admin</code> for admin users

AdminPanel.jsx — Route: `/admin`

Full whitelist management interface with data table and modal for adding users.

Column	Description
Authorized Account Name	Display name from <code>authorised_users</code>
Authentication Token	Mobile number — monospace font
System Privilege Role	Badge: indigo = admin, gray = staff
Last Verification Access	Formatted <code>toLocaleString()</code>
Verification Count	Total session count per user
Firewall Status	Toggle button: green = Authorized, red = Deactivated
Access Revocation	Delete button with confirmation dialog

AuditLog.jsx — Route: `/admin/sessions`

Full session history ledger with filtering by operator and date range.

Column	Description
Operator	Display name of the user
Verification Token	Mobile number – monospace font
Verification Login Time	Login timestamp
Session Expiry/Logout	Shows 'Active session' badge in green if is_active = true
Elapsed Duration	HH:MM:SS format, shows 'Active Operator' if still open
Network Location & Environment	IP address + truncated user-agent string

6. Request Lifecycle Walkthroughs

6.1 Full Login Flow

```
User enters mobile number on Login.jsx
|
v
POST /api/v1/auth/request-otp
|
[otpRequestLimiter] -- max 3/15min per mobile
|
requestOtp()
|-- SELECT authorised_users WHERE mobile_number = ? AND is_active = true
|-- 403 if not found
|-- crypto.randomInt(100000, 999999) -> rawOtp
|-- bcrypt.hash(rawOtp, 10) -> otpHash
|-- INSERT otp_requests (mobile_number, otp_hash, expires_at = +5min)
+-- Twilio WABA sendMessage (or console.log in dev)
|
200 OK -> frontend navigates to /verify-otp

User enters 6-digit OTP on OtpVerify.jsx
|
v
POST /api/v1/auth/verify-otp
|
[otpVerifyLimiter] -- max 5/5min per mobile
|
verifyOtpCode()
|-- SELECT otp_requests WHERE mobile=? AND is_used=false LIMIT 1
|-- Check: now() < expires_at
|-- Check: attempts < 3
|-- bcrypt.compare(rawOtp, otpHash)
|   |-- FAIL: UPDATE attempts++ -> 400 with attemptsLeft
|   +-- PASS: UPDATE is_used=true
|-- SELECT authorised_users WHERE mobile_number = ?
|-- generateToken(user) -> { token, jti }
|-- INSERT sessions (user_id, jwt_jti, ip_address, user_agent)
|-- res.cookie('token', token, { httpOnly: true, maxAge: 24h })
+-- notifyAdminLogin(...) <- async, non-blocking
|
200 OK + user data -> login(user) -> navigate('/dashboard')
```

6.2 Protected Request Flow

```
Any page with verifyJwt middleware:
|
Browser auto-sends cookie 'token' (withCredentials=true)
```

```
|  
verifyJwt()  
  |-- Extract from req.cookies.token  
  |-- jwt.verify(token, JWT_SECRET) -> decoded payload  
  |-- SELECT is_active FROM sessions WHERE jwt_jti = decoded.jti  
  |    -> 401 if missing or is_active=false  
  |-- SELECT is_active FROM authorised_users WHERE id = decoded.user_id  
  |    -> 403 if missing or is_active=false  
  +-- req.user = {...}, req.jti = jti -> next()
```

7. Security Model

Layer	Control	Implementation
Identity	Mobile number whitelist	<code>authorised_users</code> table, checked on every OTP request
Authentication	6-digit time-limited OTP	<code>bcrypt</code> -hashed, 5-min TTL, 3-attempt lockout
Delivery Security	OTP via WhatsApp (E2E encrypted)	Twilio WABA API
Token Security	<code>httpOnly</code> cookie, never in JS scope	<code>httpOnly=true</code> , <code>secure=true</code> (prod), <code>sameSite=none</code> (prod)
Token Validation	Live DB session check + JWT signature	<code>verifyJwt</code> checks both on every call
Session Invalidation	JTI-based blacklist in DB	<code>is_active=false</code> on logout/deactivation blocks token reuse
Admin Segregation	Role check in JWT + DB re-verify	<code>requireAdmin</code> middleware, role embedded in token
Rate Limiting	Per-mobile/IP throttling	<code>express-rate-limit</code> : OTP 3/15min, Verify 5/5min
DB Protection	RLS + Service Role server-side only	Client never touches DB directly
Secret Management	All credentials in <code>.env</code>	Production values injected via Render environment dashboard
HTTP Hardening	Security headers	<code>helmet()</code> on all responses
CORS	Single allowed origin	<code>FRONTEND_URL</code> env var only

8. Deployment Architecture

8.1 Live Environments

Service	Provider	URL
Frontend SPA	Vercel	https://sn-polymers.vercel.app/
Backend REST API	Render	https://snpolymers.onrender.com
Database	Supabase	(private project URL)

8.2 Architecture Diagram



8.3 Environment Variable Injection

Variable	Where Set
VITE_API_URL	Vercel -> Project Settings -> Environment Variables
All backend .env vars	Render -> Web Service -> Environment

End of Document | S.N. Polymers IDBP Phase 1 | June 2026